El algoritmo de Dijkstra

Grupo 3

19 de abril de 2026

UNIVERSIDAD DE CÓRDOBA

Integrantes del grupo

Alijo Fernández, Miguel Ángel
Buzón García, Alejandro
Casado Caballero, Antonio
Collantes Aguilar, Pablo
Deogracias Moya, Juan
Guerrero Romero, Jose Francisco
Laredo Gutiérrez, José Raúl
López Reyes, Marcos
Aramyan, David Ángel
Costa Ruiz-Canela, Javier Jesús
Ligero González, Alejandro

UNIVERSIDAD DE CÓRDOBA

Tabla de contenidos

1. Introducción
2. Descripción del algoritmo
3. Fundamento matemático
4. Ejemplo: Desarrollo paso a paso
5. Explicación del pseudocódigo
6. Simulación del algoritmo
7. Conclusiones
8. Bibliografía y webgrafía

UNIVERSIDAD DE CÓRDOBA

Introducción

Pregunta inicial

Si existen varios caminos para llegar de un punto a otro, ¿cómo podemos determinar cuál es el más eficiente?

- En muchos problemas no basta con llegar al destino.
- También interesa minimizar la distancia, el tiempo, el coste o los recursos.
- Este desafío se conoce como el **problema del camino mínimo**.

El algoritmo de Dijkstra permite abordar este problema en grafos ponderados con pesos no negativos.

UNIVERSIDAD DE CÓRDOBA

Motivación del problema

El problema del camino mínimo aparece en multitud de situaciones reales:

- **Sistemas GPS:** para calcular rutas eficientes.
- **Redes informáticas:** para enviar datos por los canales adecuados.
- **Logística y transporte:** para reducir costes y tiempos de entrega.
- **Planificación urbana:** para trazar recorridos en mapas o redes.

Por ello, su estudio es fundamental tanto en la Matemática Discreta como en la Informática.

UNIVERSIDAD DE CÓRDOBA

Objetivo del trabajo

En este trabajo analizaremos el algoritmo de Dijkstra como la técnica principal para resolver el problema del camino mínimo.

Comprender su funcionamiento nos permitirá dominar una de las herramientas más potentes y aplicadas de la teoría de grafos.

UNIVERSIDAD DE CÓRDOBA

Descripción del algoritmo

El algorimo de Dijkstra es un algorimo basado en la teoría de grafos. Un grafo, en informática es es simplemente una estructura visual que representa las relaciones entre distintos elementos y está formada por dos piezas clave:

- **vértices (o nodos)**.
- **Aristas**

Existen un tipo de grado denominado **grafo ponderado** donde , a cada arista se le asigna un valor numérico llamado **"peso"** (o coste), que indica la importancia o peso que tiene la misma.

UNIVERSIDAD DE CÓRDOBA

Descripción del algoritmo

Sobre este tipo de redes, uno de los mayores desafíos es encontrar el camino de longitud mínima entre dos ubicaciones, un proceso vital en aplicaciones cotidianas como la logística de transporte o los sistemas GPS. La solución más eficiente a este problema la aportó Edsger Dijkstra en 1956.

Su algoritmo opera bajo la condición de que todos los pesos de la red sean positivos y utiliza una estrategia progresiva de "pasos seguros" para explorar las rutas desde un origen:

UNIVERSIDAD DE CÓRDOBA

Descripción del algoritmo

1. **Inicialización**: Comienza en el punto de partida asumiendo que la distancia al resto de vértices es ïnfinita", ya que aún no ha explorado el terreno.
2. **Elección**: En cada iteración, selecciona el nodo no visitado que presenta el menor coste acumulado.
3. **Relajación**: Desde esa posición, evalúa a sus vecinos directos. Si descubre un atajo que mejora el trayecto conocido hasta ese momento, actualiza su registro con el nuevo valor mínimo.
4. **Cierre**: Tras comprobar todas sus conexiones, el lugar actual se marca como definitivo para no volver a analizarlo.

Al repetir este ciclo de manera estructurada hasta examinar todos los elementos disponibles, el método garantiza matemáticamente la obtención de la ruta óptima.

UNIVERSIDAD DE CÓRDOBA

Fundamento matemático

Para comprender el algoritmo de Dijkstra desde una perspectiva rigurosa, es necesario establecer los fundamentos que garantizan su correctitud. Esta sección presenta las definiciones formales y las propiedades estructurales en las que se basa el algoritmo.

Definición (Grafo ponderado)

Un grafo ponderado es un par (G, w), donde $G = (V, E)$ es un grafo y $w : E \to \mathbb{R}^+$ es una función que asigna pesos positivos a las aristas del grafo.

UNIVERSIDAD DE CÓRDOBA

Fundamento matemático: Camino y Distancia

Definición (Camino y su peso)

Dado un grafo ponderado (G, w) y un camino $C : v_0 v_1 \ldots v_k$, se define el peso del camino C como la suma de los pesos de las aristas que lo componen:

$$w(C) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Definición (Distancia entre vértices)

Para dos vértices distintos $u, v \in V$, la distancia $d_G(u, v)$ es:

$$d_G(u, v) = \text{mín}\{w(C) \mid C \text{ es un } u\text{-}v \text{ camino}\}$$

Si no hay camino, $d_G(u, v) = +\infty$.

Principio de optimalidad y Relajación

Principio de optimalidad de Bellman
Si un camino es mínimo, cualquier subcamino contenido en él también es un camino mínimo entre sus propios extremos.

Demostración.

Si existiera un subcamino más corto, el peso total del camino original podría reducirse, lo que contradice que era el mínimo. □

Operación de relajación
El algoritmo mantiene para cada vértice v una estimación de su distancia mínima $d[v]$ desde la fuente s (inicialmente $d[s] = 0$ y $d[v] = +\infty$ para el resto).
Para cada arista $(u, v) \in E$:

$$\text{si } d[u] + w(u, v) < d[v] \implies d[v] \leftarrow d[u] + w(u, v)$$

UNIVERSIDAD DE CÓRDOBA

Condiciones y Complejidad

Condición de pesos no negativos

Requiere que la función de peso sea $w : E \to \mathbb{R}^+$. Pesos negativos invalidarían la estrategia *greedy*, ya que una distancia "finalizada" podría reducirse posteriormente por un arco negativo.

Complejidad computacional

Estructura de datos	**Complejidad temporal**
Lista o Arreglo simple	$O(\|V\|^2)$
Heap binario	$O((\|V\| + \|E\|) \log \|V\|)$
Heap de Fibonacci	$O(\|E\| + \|V\| \log \|V\|)$

UNIVERSIDAD DE CÓRDOBA

Ejemplo: Desarrollo paso a paso

Como ejemplo del algoritmo proponemos el siguiente:

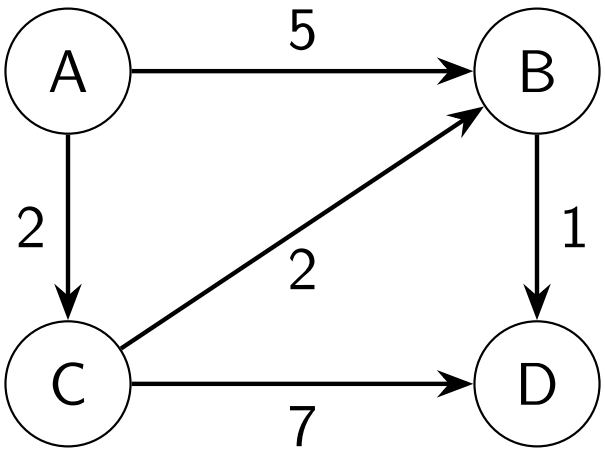
1. Definimos del grafo

   Definimos los nodos y sus conexiones (aristas) con los siguientes pesos:

   - $(A, B) \to 5$
   - $(A, C) \to 2$
   - $(B, D) \to 1$
   - $(C, B) \to 2$
   - $(C, D) \to 7$

Ejemplo: Desarrollo paso a paso

El grafo resultante nos quedaría algo así:



Ejemplo: Desarrollo paso a paso

1. Desarrollo del algoritmo (Paso a Paso)

   Estado inicial:

   - Distancias: $A = 0, B = \infty, C = \infty, D = \infty$.

     A será nuestro nodo inicial (por eso su distancia será 0), B, C, D no sabemos dónde estarán, los ponemos a distancia infinita para cubrir hasta distancias muy grandes.

   - Nodos no visitados: $\{A, B, C, D\}$.

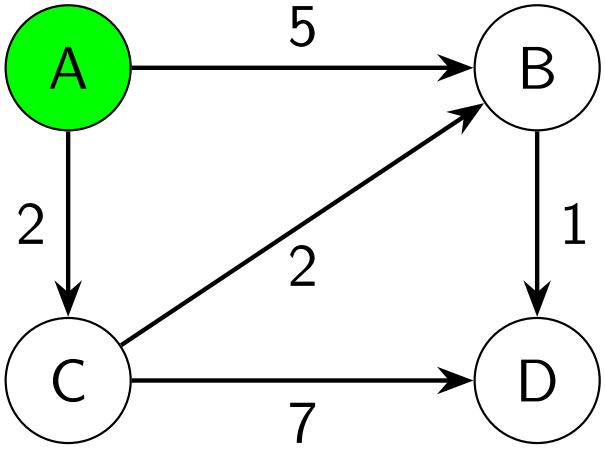
UNIVERSIDAD DE CÓRDOBA

Ejemplo: Desarrollo paso a paso

2. Desarrollo del algoritmo (Paso a Paso)

Paso 1: Seleccionamos Nodo A:

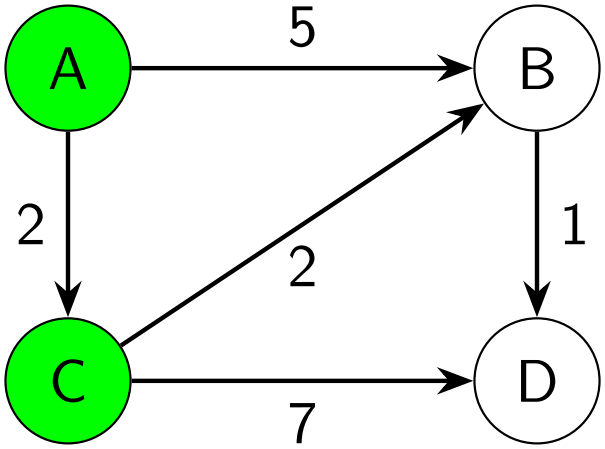
- Observamos los nodos vecinos de A: B y C.
- Para B: $0 + 5 = 5$. (Es menor que ∞ actualizamos).
- Para C: $0 + 5 = 5$. (Es menor que ∞ actualizamos).
- Para C: $0 + 2 = 2$. (Es menor que ∞ actualizamos).
- Marcamos A como visitado.



Ejemplo: Desarrollo paso a paso

Paso 2: Seleccionamos Nodo C (Distancia 2):

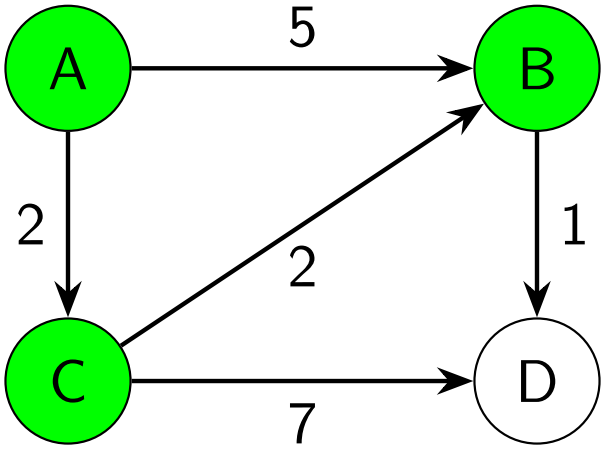
- Elegimos C porque su distancia (2) es menor que la de B (5).
- Miramos vecinos de C: B y D.
- Para B: $2 + 2 = 4$. (Como $4 < 5$, actualizamos B).
- Para D: $2 + 7 =$9. (Es menor que ∞, actualizamos D).
- Marcamos C como visitado.



Ejemplo: Desarrollo paso a paso

Paso 3: Seleccionamos Nodo B (Distancia 4):

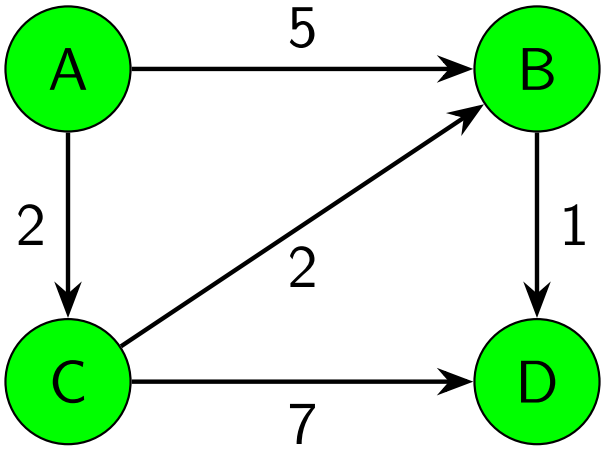
- B ahora tiene distancia 4 gracias a la actualización anterior.
- Miramos al vecino de B: D.
- Para D: $4 + 1 = 5$. (Como $5 < 9$, actualizamos D).
- Marcamos B como visitado.



Ejemplo: Desarrollo paso a paso

Paso 4: Seleccionamos Nodo D (Distancia 5):

- No tiene vecinos que no hayamos visitado.
- Marcamos D como visitado.



Ejemplo: Desarrollo paso a paso

3. Resumen de Resultados para el Trabajo

Nodo	**Distancia mínimia nodo A**	**Camino (Predecesores)**
A	0	-
B	4	$A \to C \to B$
C	2	$A \to C$
D	5	$A \to C \to B \to D$

UNIVERSIDAD DE CÓRDOBA

Ejemplo: Desarrollo paso a paso

4. Conclusiones

   Para encontrar la ruta más corta en esté grafo, se ha aplicado el algoritmo de Dijkstra de la siguiente forma: Inicialmente, el camino directo de A a B tenía un coste de 5, pero evaluando el nodo C, podemos apreciar una ruta alternativa $(A - C - B)$ con un coste de 4. Finalmente, para llegar a D, la ruta óptima no es la directa desde C (coste 9), es la que pasa por B, logrando un coste total de 5.

Pseudocódigo del algoritmo

```
function Dijkstra(Graph, source):
   dist[source] = 0
   for each vertex v in Graph:
       if v != source:
           dist[v] = infinity
       add v to Q (unvisited set)
   while Q is not empty:
       u = vertex in Q with min dist[u]
       remove u from Q
       for each neighbor v of u in Q:
           alt = dist[u] + length(u, v)
           if alt < dist[v]:
               dist[v] = alt
               prev[v] = u
   return dist, prev
```

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo

El algoritmo de Dijkstra calcula la ruta más corta desde un nodo inicial hacia todos los demás nodos del grafo. Requiere que todas las distancias sean positivas o cero. Su idea principal es seleccionar siempre el nodo no visitado más cercano al origen y actualizar las distancias de sus nodos vecinos si encuentra un camino más corto pasando por él.

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo: línea por línea I

- `dist[source] = 0`: La distancia para ir del punto de inicio a sí mismo es cero.
- `for each vertex v in Graph`: Recorre todos los demás puntos o nodos de la red.
- `dist[v] = infinity`: Como al principio el algoritmo no sabe cómo llegar al resto de nodos, les asigna temporalmente una distancia infinita.
- `add v to Q`: Introduce todos los nodos en un conjunto llamado `Q`, que funciona como una lista de tareas de nodos no visitados.

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo: línea por línea II

- `while Q is not empty`: El proceso se repite en bucle hasta que la lista de nodos no visitados quede vacía.
- `u = vertex in Q with min dist[u]`: Selecciona de la lista el nodo que tenga la menor distancia acumulada. En la primera iteración, este será necesariamente el nodo origen.
- `remove u from Q`: Elimina ese nodo de la lista para marcarlo oficialmente como visitado.
- `for each neighbor v of u in Q`: Revisa únicamente los nodos vecinos de `u` que todavía no hayan sido visitados.

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo: línea por línea III

- `alt = dist[u] + length(u, v)`: Calcula la longitud de una posible nueva ruta hasta `v`, sumando la distancia ya conocida hasta `u` más el peso de la arista entre `u` y `v`.
- `if alt < dist[v]`: Este es el núcleo del algoritmo. Comprueba si la nueva distancia calculada es menor que la mejor distancia conocida hasta ese momento para llegar a `v`.
- `dist[v] = alt` y `prev[v] = u`: Si la nueva ruta es mejor, se actualiza la distancia mínima registrada y se guarda que el nodo anterior óptimo para llegar a `v` es `u`.

UNIVERSIDAD DE CÓRDOBA

Resultado final del algoritmo

- `return dist, prev`: El algoritmo finaliza devolviendo dos estructuras:
  - `dist`, que contiene la distancia mínima desde el nodo origen hasta cada nodo del grafo.
  - `prev`, que permite reconstruir el camino más corto seguido para llegar a cada nodo.

Gracias a estas dos estructuras, no solo se conoce cuánto cuesta llegar a cada punto, sino también cuál es la ruta exacta que debe seguirse.

UNIVERSIDAD DE CÓRDOBA

Simulación del algoritmo

En el siguiente enlace encontrarás una simulación **desarrollada por nosotros** del algoritmo programa en `react.js`:

Pulsa este enlace para acceder a la simulación

UNIVERSIDAD DE CÓRDOBA

Conclusiones

Bibliografía y webgrafía

- **Sitio web (Artículo de blog)**: Byte, L. (2024, 24 de octubre). El Algoritmo de Dijkstra: Qué es, Cómo Funciona y Dónde se utiliza. CodigoNautas. https://codigonautas.com.
- **Video de YouTube**: Gomez, J. A. (2012, 6 de agosto). Algoritmo de Dijkstra (1) - Teoría de Grafos [Video]. YouTube. https://www.youtube.com.

UNIVERSIDAD DE CÓRDOBA